

Low-Level Design (LLD)

RAG-Based Customer Support Assistant

1. Module-Level Design

1.1 Document Processing Module

File: ingest.py

Responsibilities:

- Validate PDF file path
 - Load PDF using PyPDFLoader
 - Extract and organize document metadata
-

1.2 Chunking Module

File: ingest.py

Description:

- Splits document into smaller chunks
- Uses RecursiveCharacterTextSplitter

Input: Full document text

Output: Chunked text with overlap

1.3 Embedding Module

File: ingest.py

Function: build_embeddings()

Responsibilities:

- Convert text chunks into vectors
 - Uses Sentence Transformer model
-

1.4 Vector Storage Module

Files: ingest.py, retrieval.py

Responsibilities:

- Store embeddings in ChromaDB
 - Maintain metadata for retrieval
-

1.5 Retrieval Module

File: retrieval.py

Class: KnowledgeRetriever

Function:

- Retrieve top-k relevant chunks
 - Combine results into context
 - Provide confidence score
-

1.6 Query Processing Module

File: routing.py

Responsibilities:

- Detect user intent
 - Analyze query complexity
 - Decide routing (answer or escalate)
-

1.7 Graph Execution Module

File: workflow.py

Class: SupportAssistant

Responsibilities:

- Execute workflow using LangGraph
 - Manage state between nodes
-

1.8 Human-in-the-Loop (HITL) Module

File: hitl.py

Responsibilities:

- Handle escalated queries
 - Accept human response via CLI
-

2. Data Structures

2.1 Document Structure

```
Document {  
  page_content: string,  
  metadata: {  
    page: int,  
    source: string,  
    chunk_index: int  
  }  
}
```

2.2 Chunk Format

```
{  
  chunk_id: string,  
  page: int,  
  score: float,  
  content: string  
}
```

2.3 Embedding Structure

- Dense numerical vector
 - Stored in ChromaDB with metadata
-

2.4 Query-Response Format

```
{  
  query: string,  
  intent: string,
```

```
route: string,  
confidence: float,  
answer: string  
}
```

2.5 Graph State Object

Contains:

- query
 - intent
 - route
 - confidence
 - retrieved_chunks
 - context
 - answer
 - human_response
-

3. Workflow Design (LangGraph)

Nodes

- **Input Node** → Accepts user query
 - **Processing Node** → Retrieval + routing logic
 - **Output Node** → Generates answer or triggers HITL
-

Edges

START → Processing Node

Processing → Output Node

Output → END

State Flow

- Query enters system

- Context is retrieved
 - Decision is made
 - Output is generated
-

4. Conditional Routing Logic

Auto Answer Conditions

- Relevant context is available
 - Confidence score is high
 - Query is simple
-

Escalation Conditions

- No relevant data found
 - Low confidence score
 - Complex or sensitive query
-

Routing Logic (Simplified)

If no data → Escalate

If low confidence → Escalate

If complex query → Escalate

Else → Generate Answer

5. Human-in-the-Loop (HITL)

When Triggered

- System cannot confidently answer

Process

1. Query is forwarded to human
2. Human provides response
3. Response returned to user

6. Interface Design

CLI Commands

Ingest

```
python app.py ingest --pdf file.pdf
```

Ask

```
python app.py ask "your question"
```

Chat

```
python app.py chat
```

7. Error Handling

- Missing PDF → error message
 - No relevant data → escalate
 - LLM failure → fallback to HITL
-

8. Testing Approach

- Test normal queries
- Test unknown queries
- Test escalation cases